Machine Learning

K-Nearest Neighbours Classification

Tarapong Sreenuch

8 February 2024

克明峻德，格物致知

We are a bookstore owner stocking up on new releases.

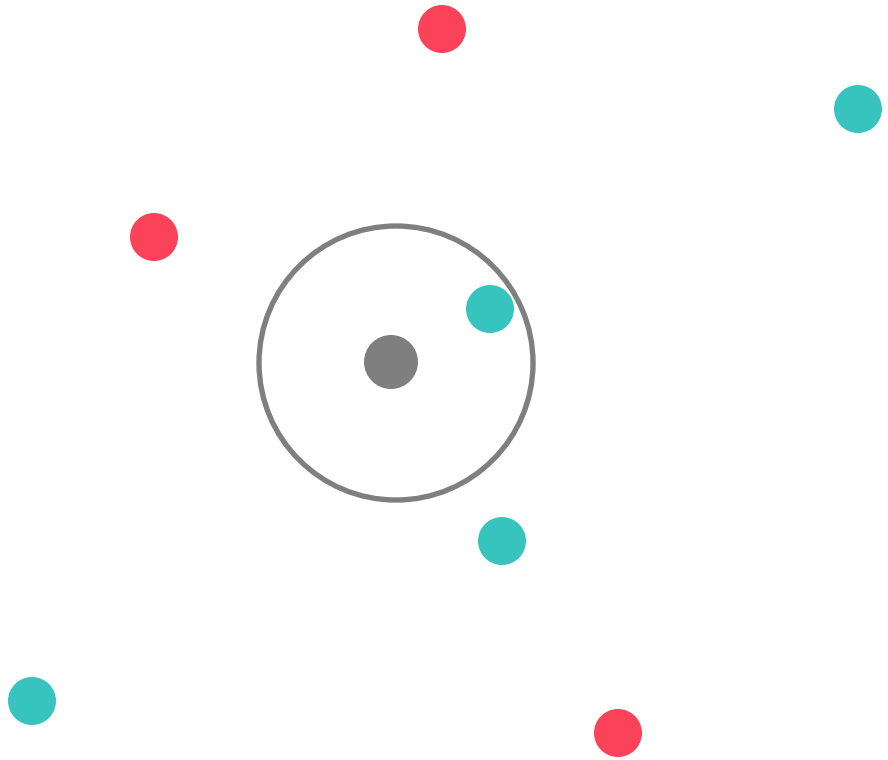
Certain authors and genres have historically achieved bestseller status more frequently than others.

Q: How do we decide which books might become bestsellers?

A: We assess recent genre trends and author track records during similar seasons to predict which books will top the market. the market.

K-Nearest Neighbours

The KNN algorithm predicts the labels of the test dataset by looking at the labels of its closest neighbors in the feature space of the training dataset. The “K” is the most important hyperparameter that can be tuned to optimize the performance of the model.



K = 1

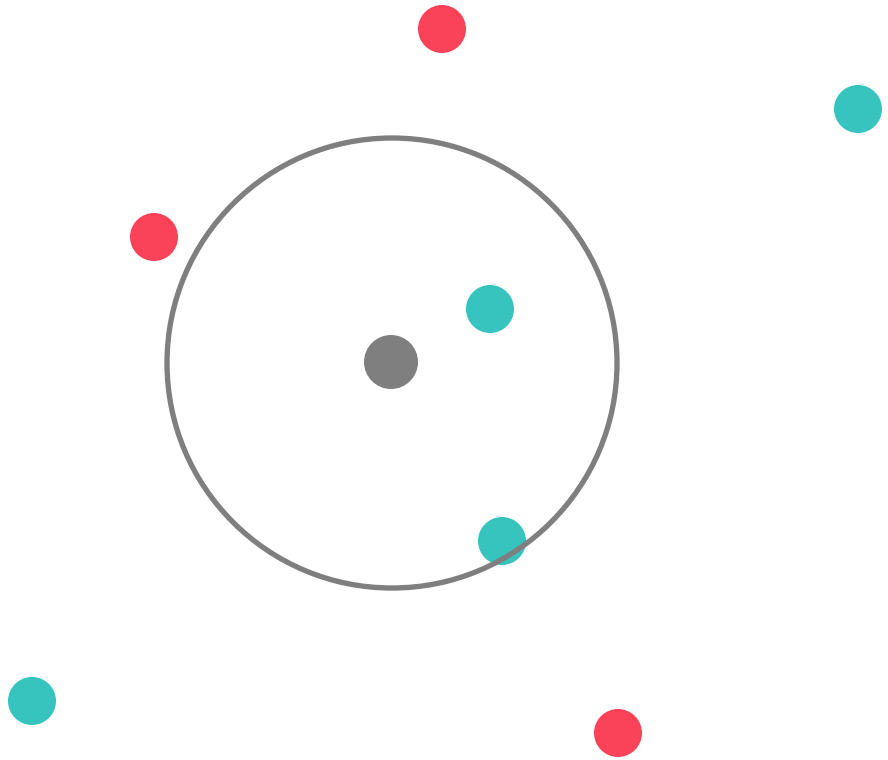
Green = 1

Red = 0

Prediction: **Green**

K-Nearest Neighbours

The KNN algorithm predicts the labels of the test dataset by looking at the labels of its closest neighbors in the feature space of the training dataset. The “K” is the most important hyperparameter that can be tuned to optimize the performance of the model.



K = 2

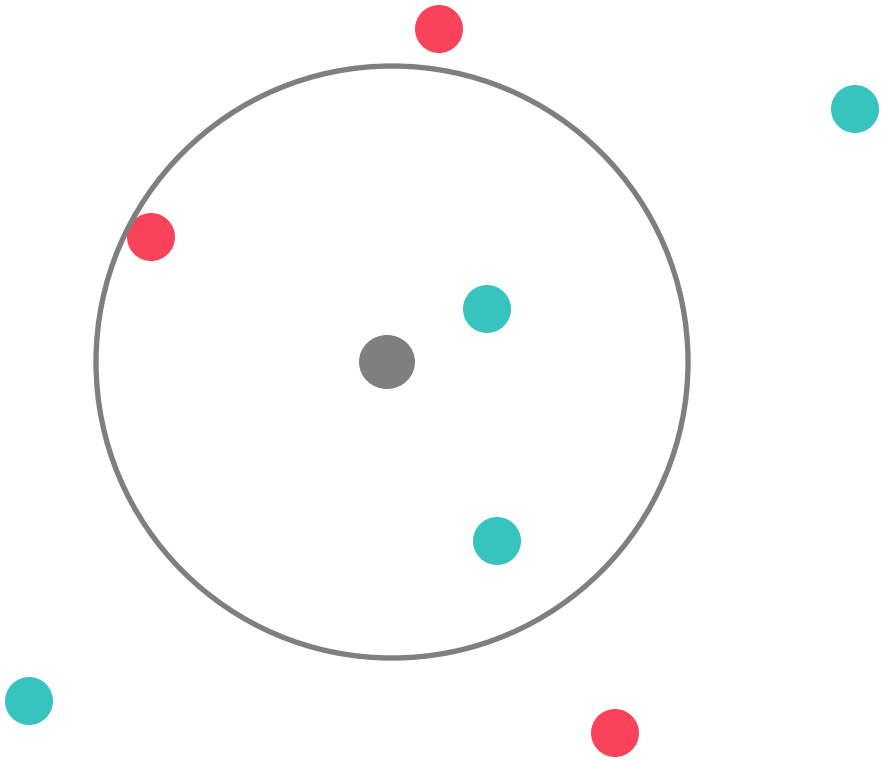
Green = 2

Red = 0

Prediction: **Green**

K-Nearest Neighbours

The KNN algorithm predicts the labels of the test dataset by looking at the labels of its closest neighbors in the feature space of the training dataset. The “K” is the most important hyperparameter that can be tuned to optimize the performance of the model.



K = 3

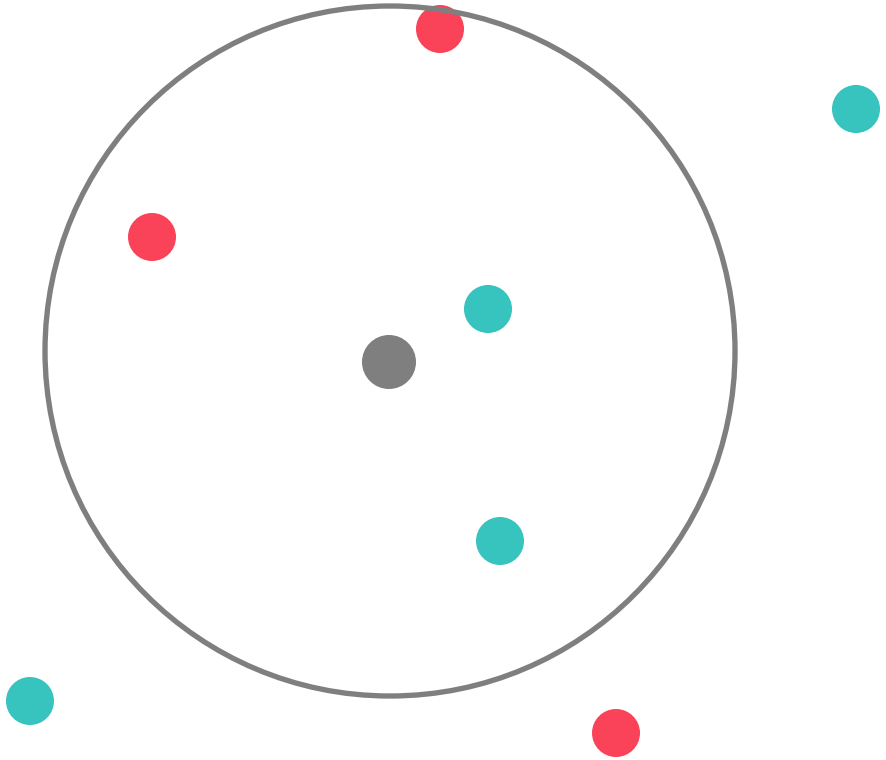
Green = 2

Red = 1

Prediction: **Green**

K-Nearest Neighbours

The KNN algorithm predicts the labels of the test dataset by looking at the labels of its closest neighbors in the feature space of the training dataset. The “K” is the most important hyperparameter that can be tuned to optimize the performance of the model.



K = 4

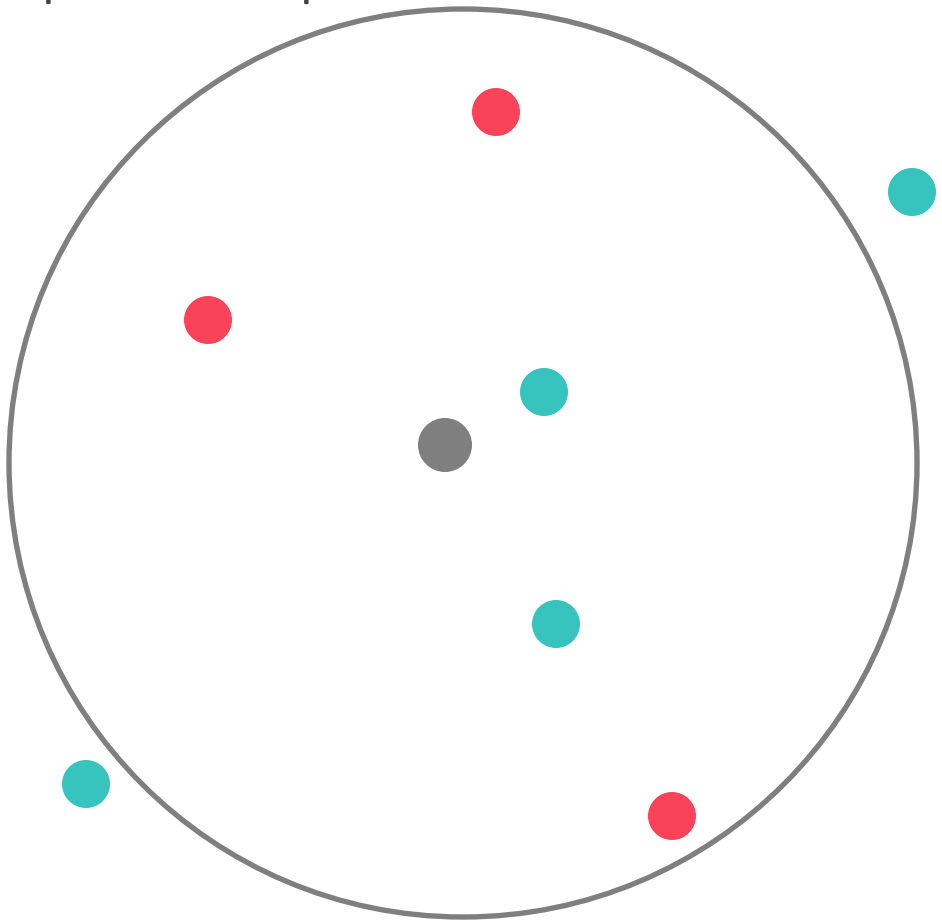
Green = 2

Red = 2

Prediction: It’s a tie.

K-Nearest Neighbours

The KNN algorithm predicts the labels of the test dataset by looking at the labels of its closest neighbors in the feature space of the training dataset. The “K” is the most important hyperparameter that can be tuned to optimize the performance of the model.



K = 5

Green = 2

Red = 3

Prediction: **Red**

Pseudocode for kNN Classifier

```
Function kNN_Classify(new_point, data_points, labels, k)
  Begin
    // Step 1: Calculate distances
    Initialize an empty list: distances
    For each point in data_points do
      Calculate the distance between new_point and point
      Add the distance and the corresponding label to the distances list

    // Step 2: Sort distances
    Sort distances in ascending order (based on distance values)

    // Step 3: Select k nearest neighbors
    Select the first k elements from the sorted distances list

    // Step 4: Perform majority vote
    Initialize an empty dictionary: class_votes
    For each neighbor in the k nearest neighbors do
      If neighbor's class in class_votes then
        Increment count of the class in class_votes
      Else
        Add the class to class_votes with count 1

    // Step 5: Determine the predicted class
    predicted_class = class with the highest count in class_votes

    Return predicted_class
  End
```

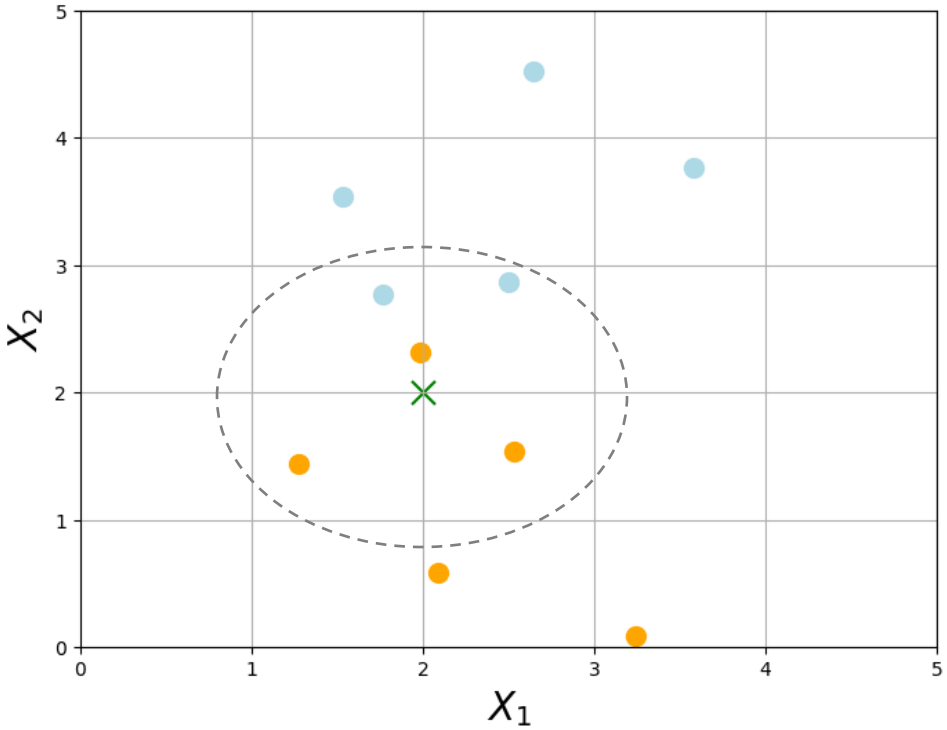
Python Code Snippet

```
import numpy as np
from sklearn.metrics import pairwise_distances
from collections import Counter

def kNN_Classify(new_point, data_points, labels, k, distance_metric='euclidean'):
    # Step 1: Calculate distances using pairwise_distances
    distances = pairwise_distances(data_points, [new_point], metric=distance_metric).ravel()

    # Step 2 & 3: Sort distances and select k nearest neighbors
    k_nearest_neighbors_indices = np.argsort(distances)[:k]
    k_nearest_labels = labels[k_nearest_neighbors_indices]

    # Step 4: Perform majority vote
    most_common = Counter(k_nearest_labels).most_common(1)

    # Step 5: Determine the predicted class
    predicted_class = most_common[0][0]

    return predicted_class
```

Usage Example

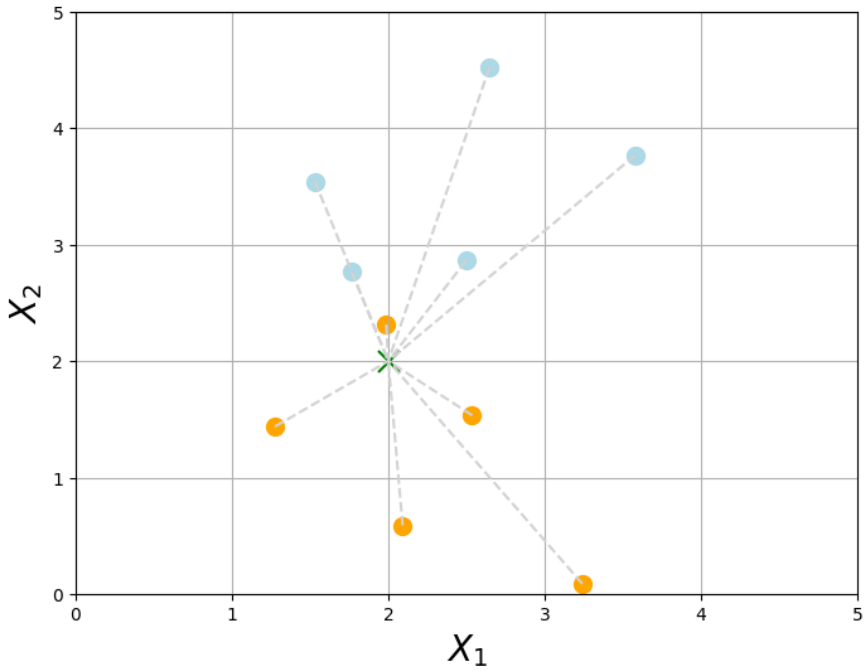
```
# Example usage
k = 5
distance_metric = 'euclidean'  # Can be 'euclidean', 'manhattan', 'cosine', etc.

# Classify the new point
predicted_class = kNN_Classify(new_point, data_points, labels, k, distance_metric)
print("Predicted Class:", predicted_class)
```

```
Predicted Class: 1.0
```

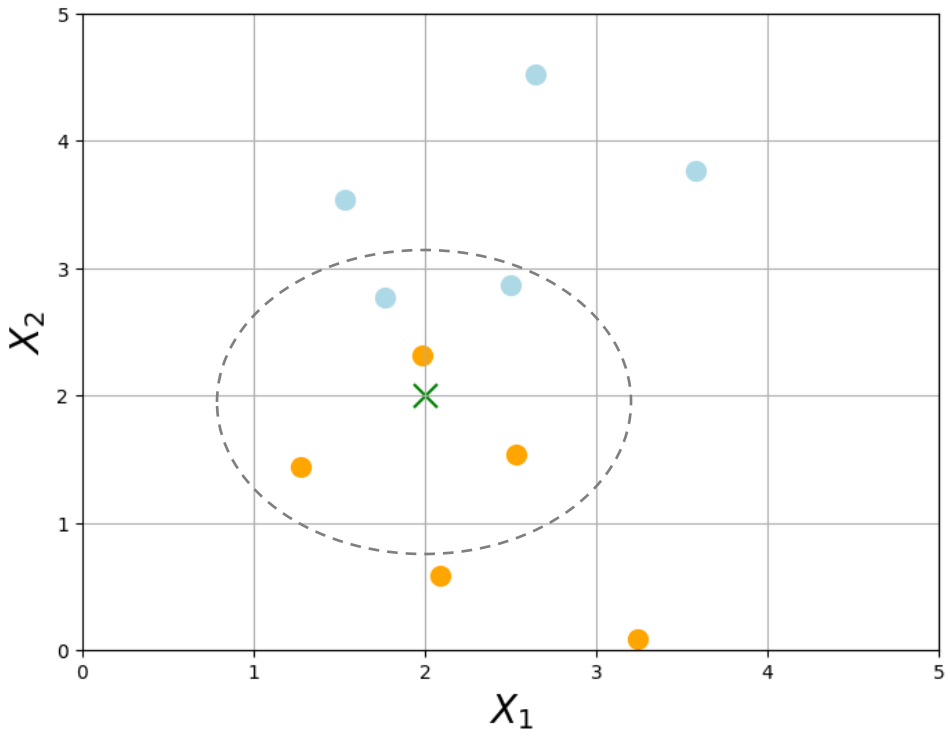


Step 1: Pairwise Distance Calculation

```
# Step 1: Calculate distances using pairwise_distances
distances = pairwise_distances(data_points, [new_point], metric=distance_metric).ravel()
```



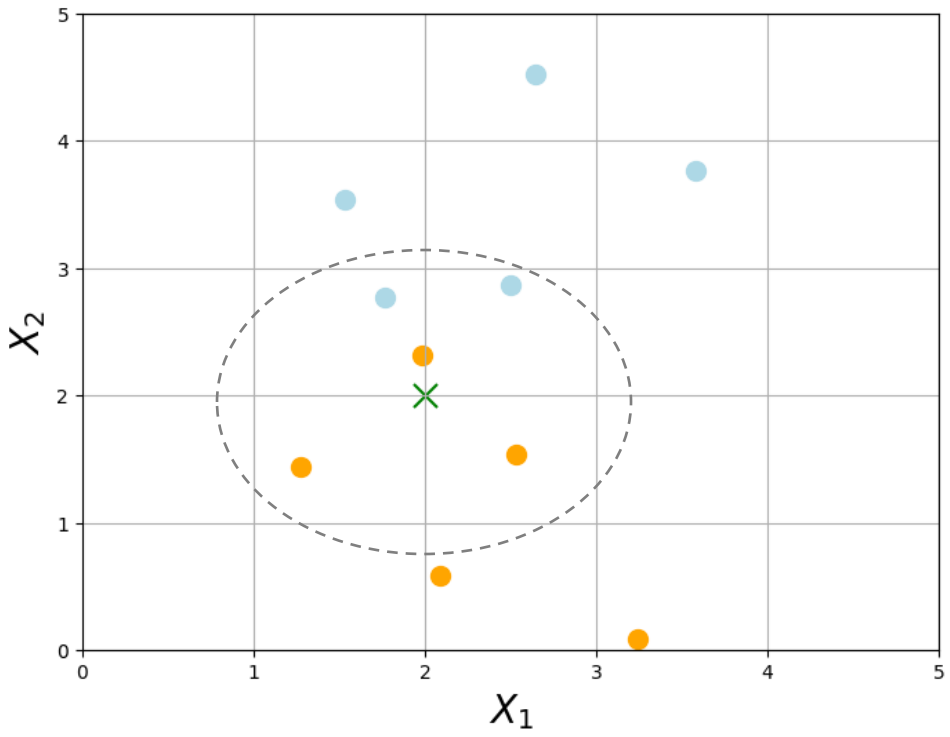
Step 2 & 3: Finding k Nearest Neighbours




```
# Step 2 & 3: Sort distances and select k nearest neighbors
k_nearest_neighbors_indices = np.argsort(distances)[:k]
k_nearest_labels = labels[k_nearest_neighbors_indices]
```

K = 5

Step 4 & 5: Prediction based on Majority Voting




```
# Step 4: Perform majority vote
most_common = Counter(k_nearest_labels).most_common(1)

# Step 5: Determine the predicted class
predicted_class = most_common[0][0]
```

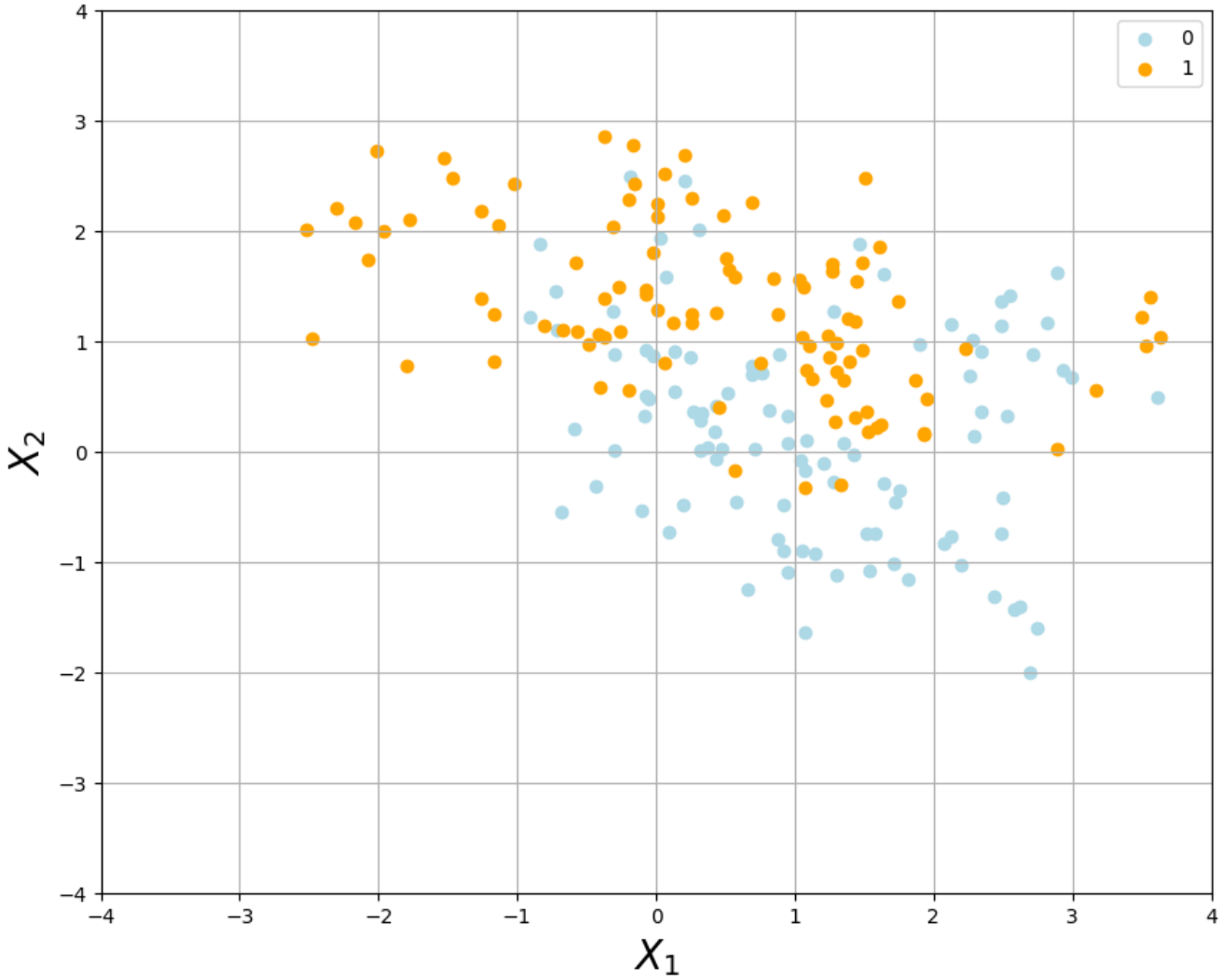
K = 5

Blue = 2

Orange = 3

Prediction: **Orange**

R Mixture Example Dataset



ElemStatLearn Package:

- Simulated Mixture Guassian Dataset
- 2 Features, 2 Classes

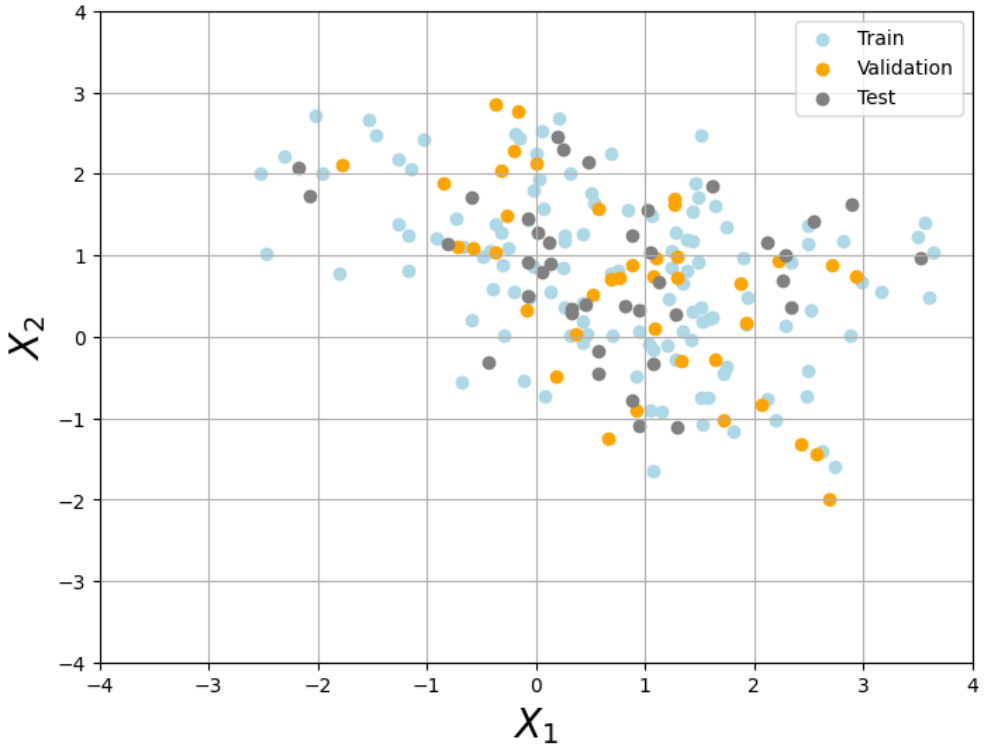
What Makes a Good ML Model?

- We learn through tutorials and practice problems. Hopefully, we develop understanding of the subject.
- To pass an exam, we are likely to be tested on unseen exam questions, so memorising the practice ones will not work.

- Like wise, a good classifer is the one that consistently performs well on both trainning data and also an unseen dataset.
- The model must learn underlying relationships in the data, instead of memorising it.

Train, Validation and Test Datasets

```
from sklearn.model_selection import train_test_split

# First, split the data into train (60%) and temp (40%)
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.4, random_state=42, stratify=y)

# Then split the temp data into validation (50% of 40% => 20% of total) and test (remaining 50% of 40% => 20% of total)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, random_state=42, stratify=y_temp)
```



Train	Validatio	Test

- (Train+Validation):Test is typicaly either 0.8:0.2 or 0.7:0.3.
- **Test** dataset is a proxy of unseen data, and it will only be used in the final evaluation.
- We train our ML model on the **Train** dataset.
- **Validation** datest is used to fine-tune or optimise the ML model. Here, we find a set of hyper-parameters, e.g. k in the kNN context, that will result in the best performance on the Validation dataset.

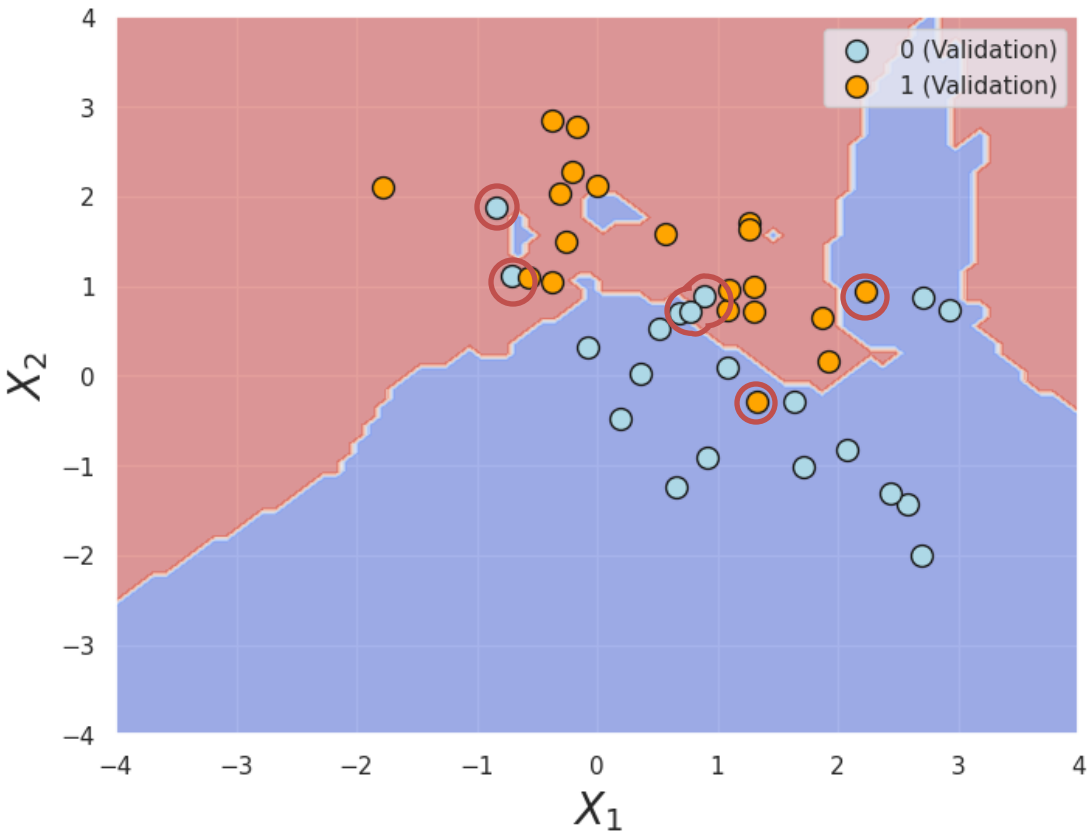
Train, Validation and Test Datasets (cont.)

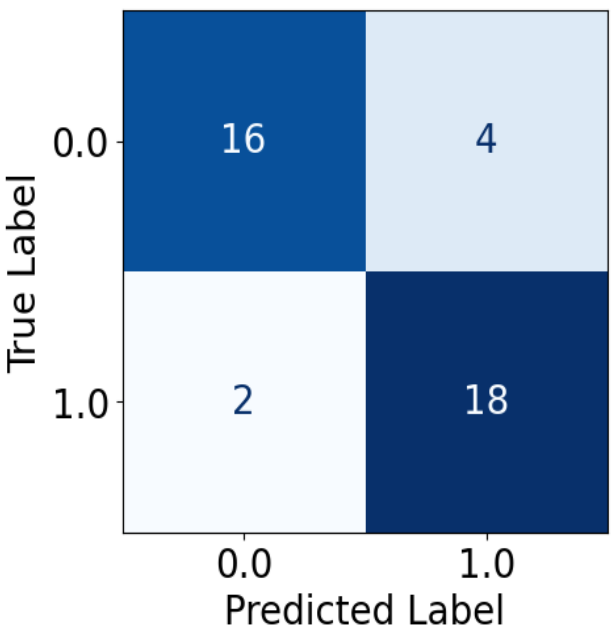
scikit-learn: KNeighborsClassifier

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Define k and the distance metric
k = 5
distance_metric = 'euclidean'

# Create an instance of KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=k, metric=distance_metric)

# Fit the model on the training data
knn.fit(X_train, y_train)

# Predict the labels on the validation set
predictions = knn.predict(X_val)

# Calculate the accuracy of the predictions
accuracy = accuracy_score(y_val, predictions)
print("Accuracy of the k-NN classifier on the validation set:", accuracy)
```

```
Accuracy of the k-NN classifier on the validation set: 0.85
```

Decision Boundary





$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

$$= \frac{4+2}{16+4+2+18}$$

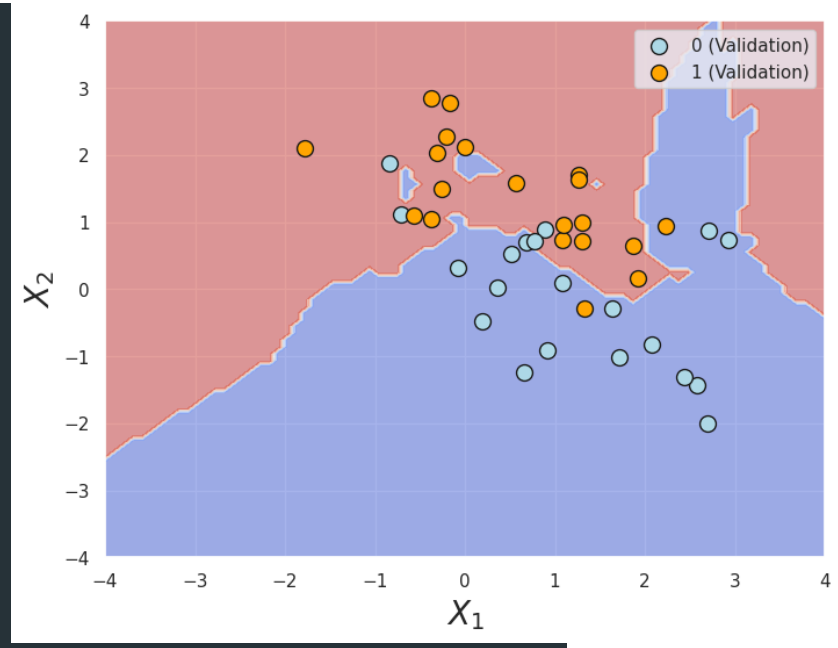
$$= 0.85$$

MyKNNClassifier (Revised)

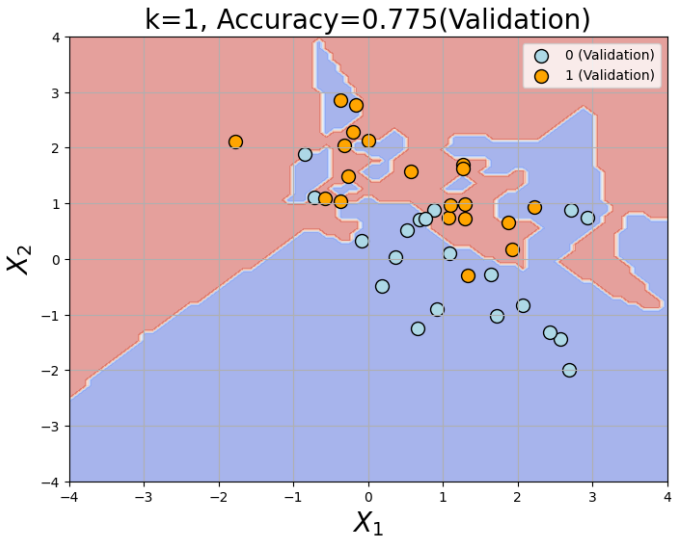
```
from sklearn.base import BaseEstimator, ClassifierMixin

class MyKNNClassifier(BaseEstimator, ClassifierMixin):
    def __init__(self, k=3, distance_metric='euclidean'):
        self.k = k
        self.distance_metric = distance_metric

    def fit(self, X, y):
        self.X_train_ = X
        self.y_train_ = y
        return self

    def predict(self, X):
        # Your custom predict logic here
        y_pred = []
        for x in X:
            # Calculate distances
            distances = pairwise_distances(self.X_train_, [x], metric=self.distance_metric).ravel()

            # Get indices of k nearest neighbors
            k_nearest_indices = np.argsort(distances)[:self.k]
            k_nearest_labels = self.y_train_[k_nearest_indices]

            # Perform majority vote
            most_common = Counter(k_nearest_labels).most_common(1)
            predicted_class = most_common[0][0]
            y_pred.append(predicted_class)

        return np.array(y_pred)
```

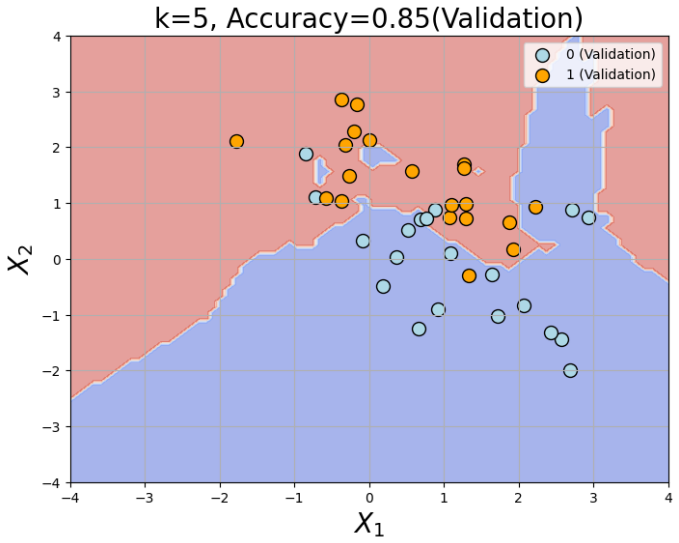
Decision Boundary

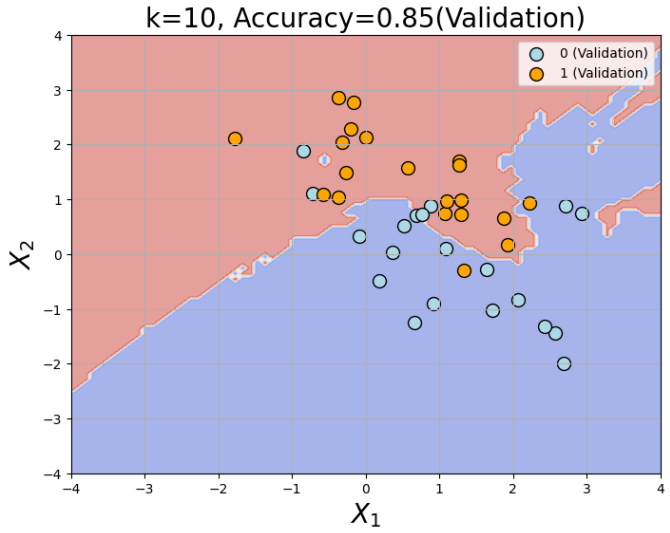
```
from sklearn.inspection import DecisionBoundaryDisplay

# Train the classifier
knn = MyKNNClassifier(k=5, distance_metric="euclidean")
knn.fit(X_train, y_train)

# Plotting decision boundaries
fig, ax = plt.subplots(figsize=(8, 6))
db_display = DecisionBoundaryDisplay.from_estimator(
    estimator=knn, X=X_train, response_method="predict",
    alpha=0.5, ax=ax, cmap='coolwarm'
)

# Scatter plot of the validation data with class labels
class_names = ['0', '1']
colors = ['lightblue', 'orange']  # Define colors for different classes
for i, color in enumerate(colors):
    idx = np.where(y_val == i)
    ax.scatter(X_val[idx, 0], X_val[idx, 1], c=color, edgecolor='k', s=100, label=f'{class_names[i]} (Validation)')

# Adding labels, title and custom legend for validation data
ax.set_xlabel('$X_1$', fontsize=20)
ax.set_ylabel('$X_2$', fontsize=20)
ax.legend(loc='best')  # Position the legend to not overlap with data
ax.grid()
plt.axis([-4, 4, -4, 4])
plt.show()
```

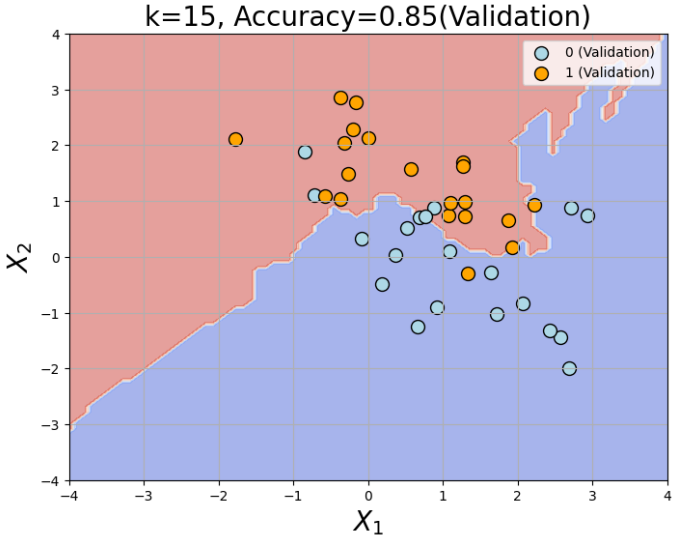


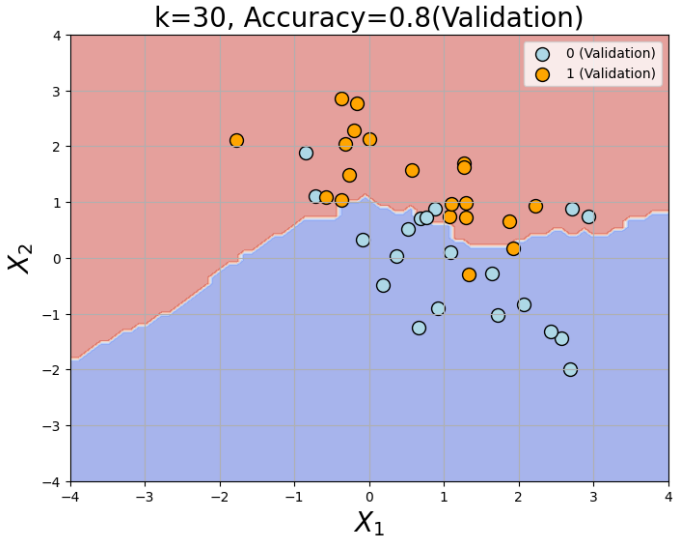
Model Complexity

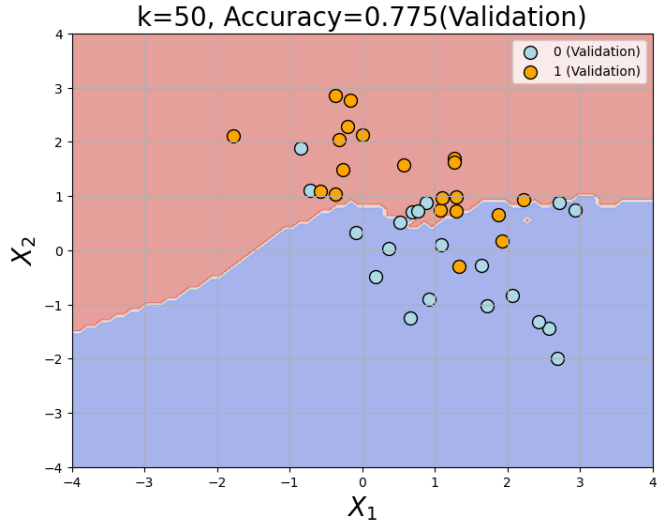




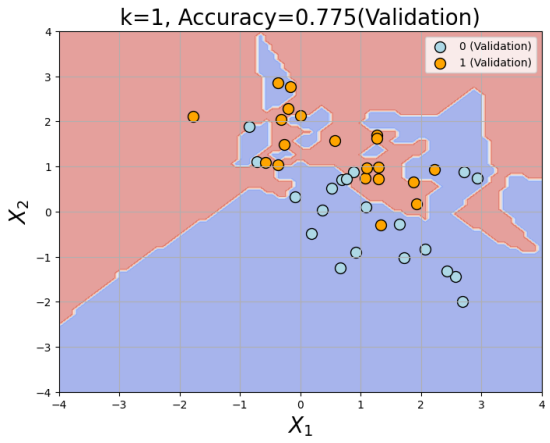




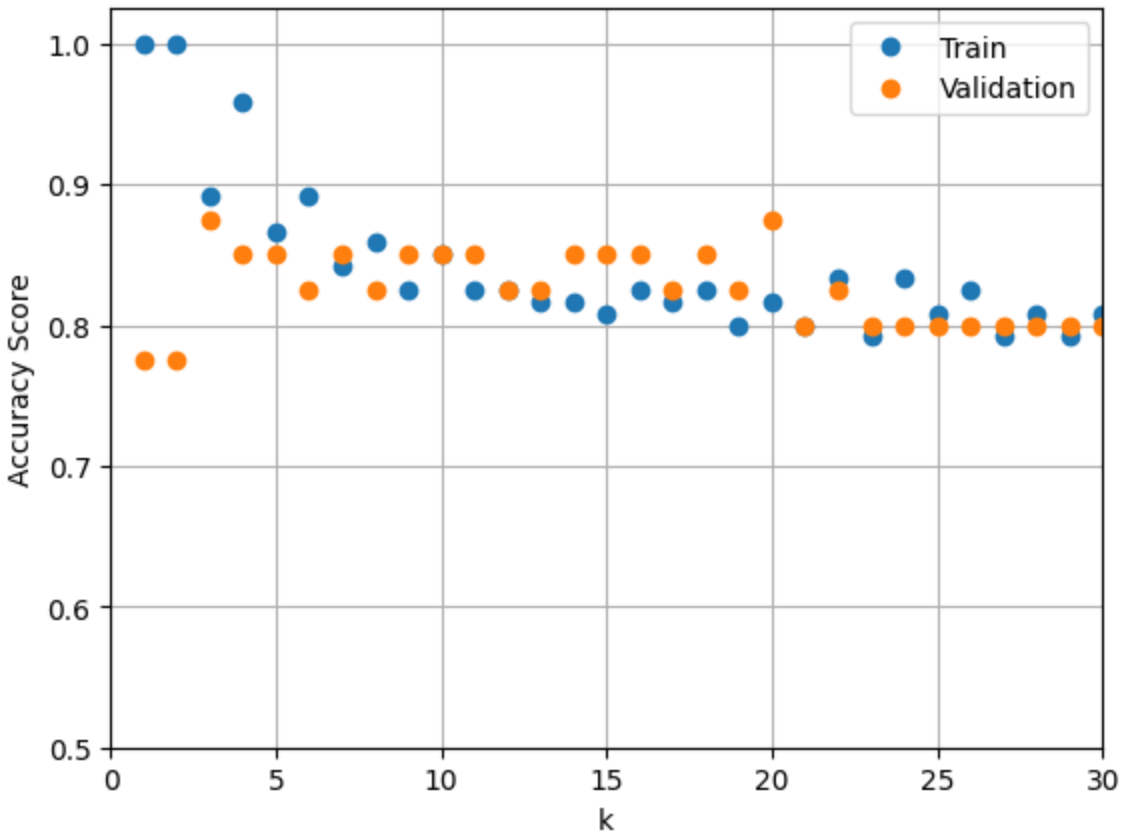


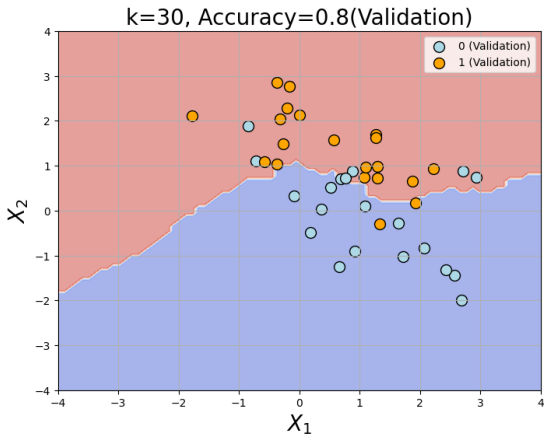


Bias-Variance Trade-Offs



- High Variance, Low Bias
- Complex Decision Boundary
- High Train Accuracy
- Low Validation Accuracy

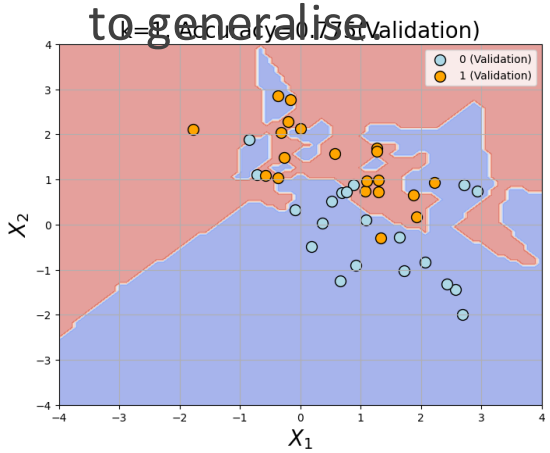


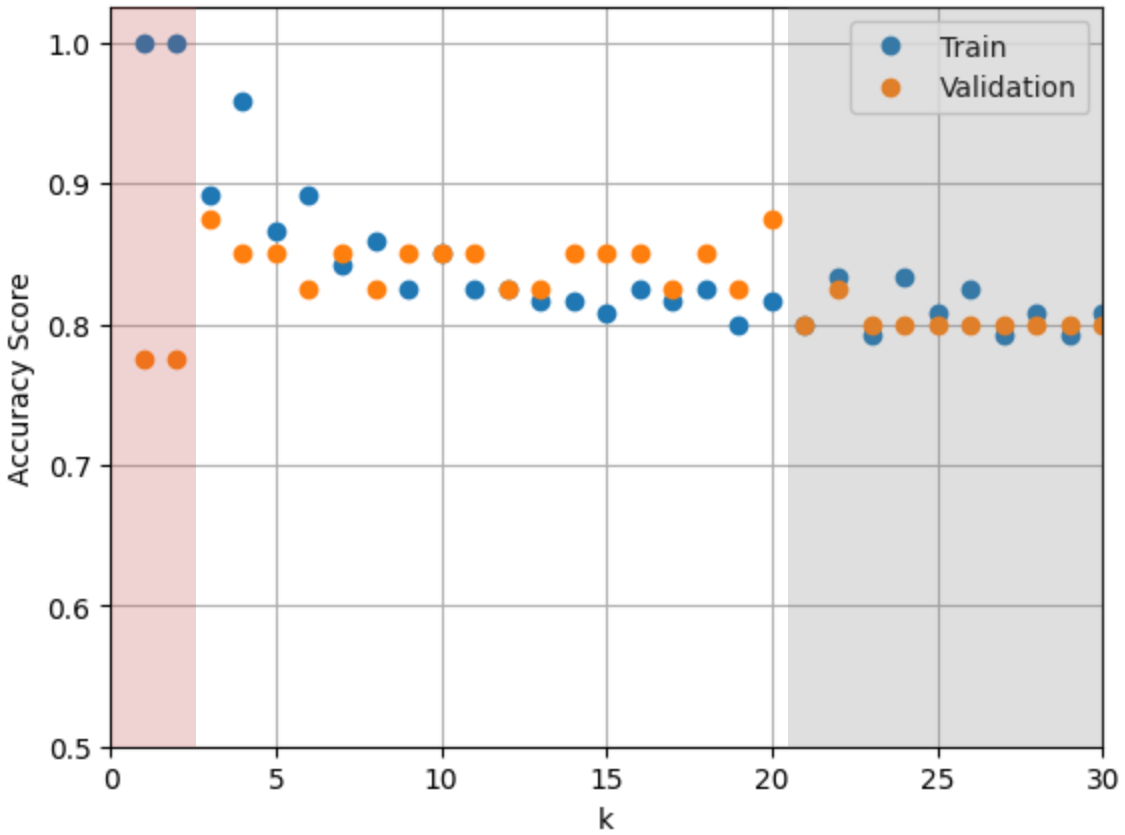


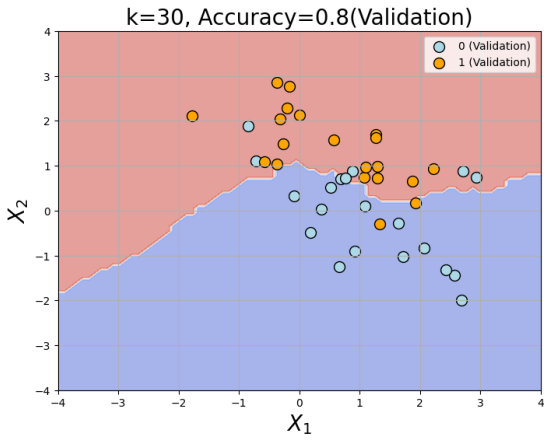
- High Bias, Low Variance
- Simple Decision Boundary
- Low Train Accuracy
- Low Validation Accuracy

Overfitting vs Underfitting

- **Overfitting** happens when a small k leads to high training accuracy but poor validation accuracy due to overly complex decision boundaries that fail to generalise.
- **Underfitting** occurs when a large k leads to overly simple decision boundaries, resulting in low training and validation accuracies.tion accuracy due to overly complex decision boundaries that fail to generalise.





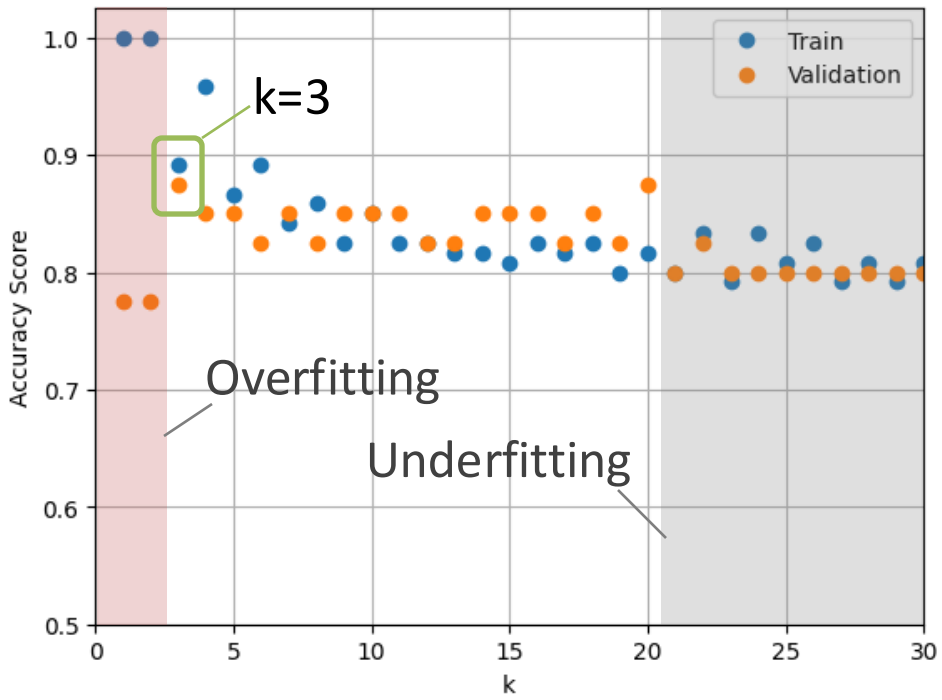


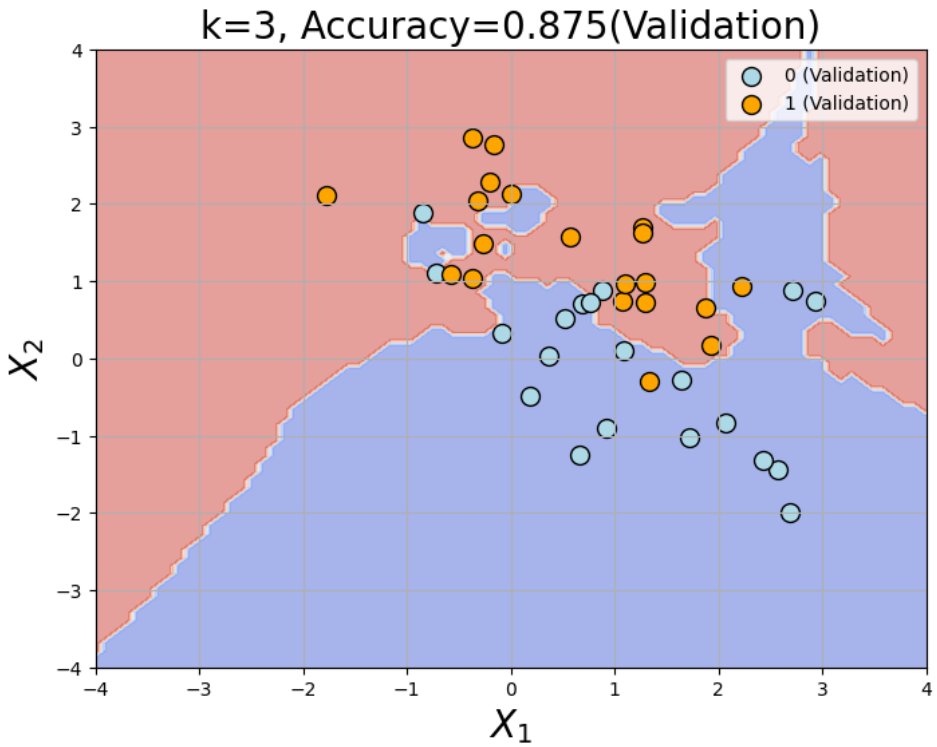
- High Variance, Low Bias
- Complex Decision Boundary
- High Train Accuracy
- Low Validation Accuracy

- High Bias, Low Variance
- Simple Decision Boundary
- Low Train Accuracy
- Low Validation Accuracy

Optimal Model

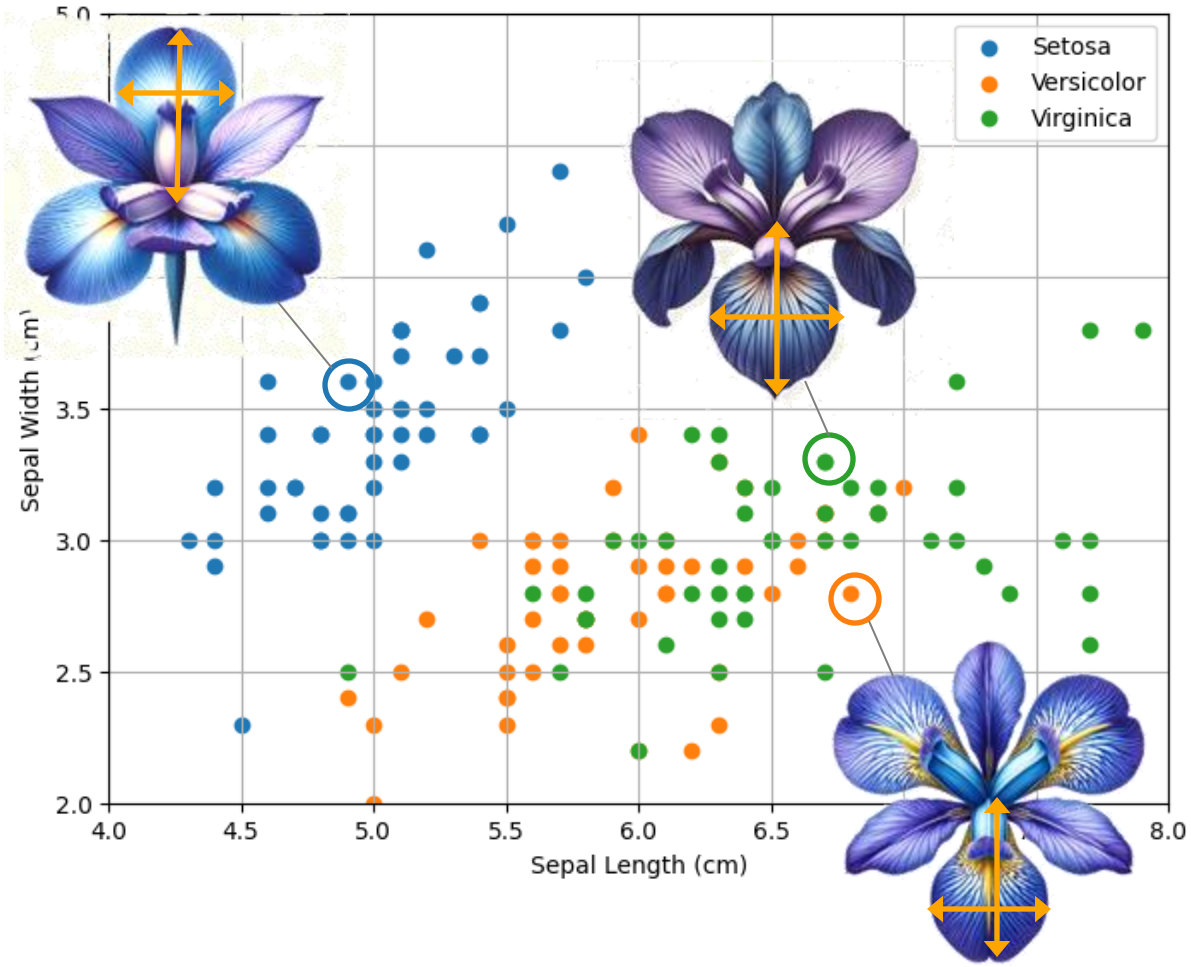
- An optimal k-NN model is one where the choice of k balances the complexity of decision boundaries to achieve high validation accuracy while maintaining good training accuracy, indicating effective generalisation without overfitting or underfitting.





What is Nearest?

- For kNN to function, we must define the notation of ‘Nearest’.
- A: Distance (or Similarity) Measure



Versicolor ⇔ Virginica

- Less Distance implies Near (High Similarity).

Setosa ⇔ Virginica

- Further Distance implies Far (Low Similarity).

What is Nearest? (cont.)

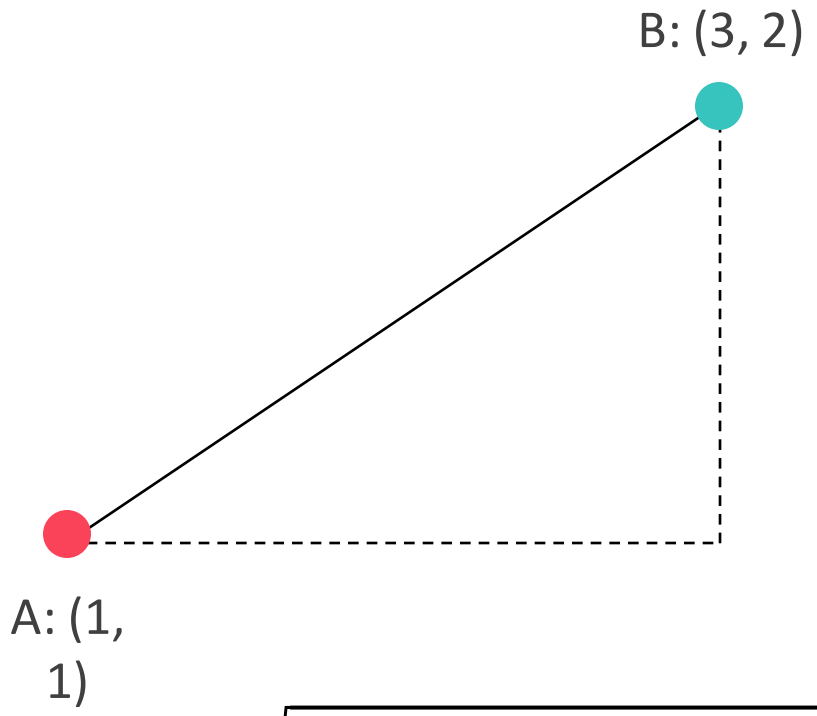
- For kNN to function, we must define the notation of ‘Nearest’.
- A: Distance (or Similarity) Measure

High Similarity implies Near (or Less Distance).

Low Similarity implies Far (or Further Distance).

Distance Metrics

Euclidean Distance (Norm 2)

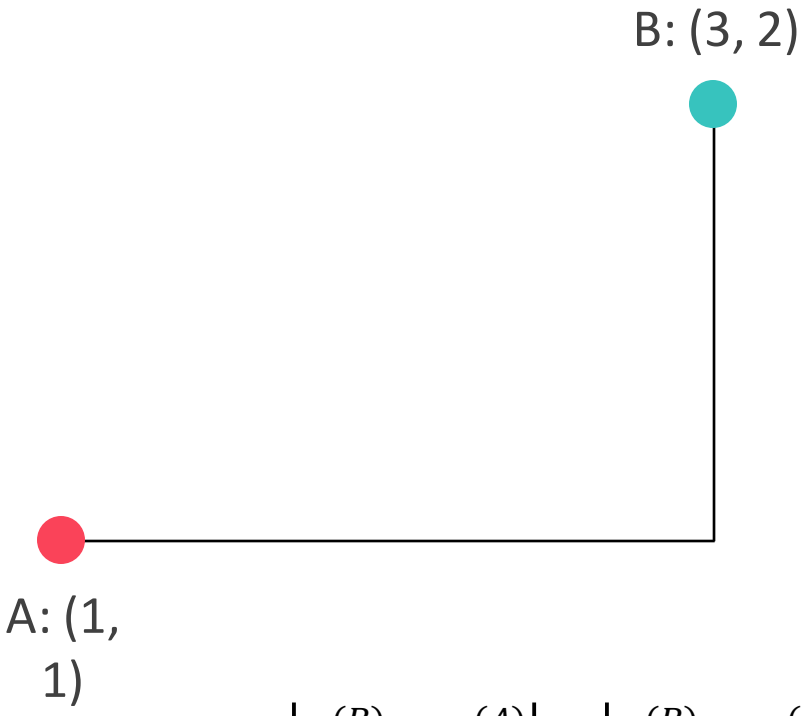


$$\|A-B\|_2 = \sqrt{(x_1^{(B)}-x_1^{(A)})^2+(x_2^{(B)}-x_2^{(A)})^2}$$

$$= \sqrt{(3-1)^2+(2-1)^2}$$

$$= \sqrt{2^2+1^2} = \sqrt{5}\ (\sim 2.24)$$

Manhattan Distance (Norm 1)

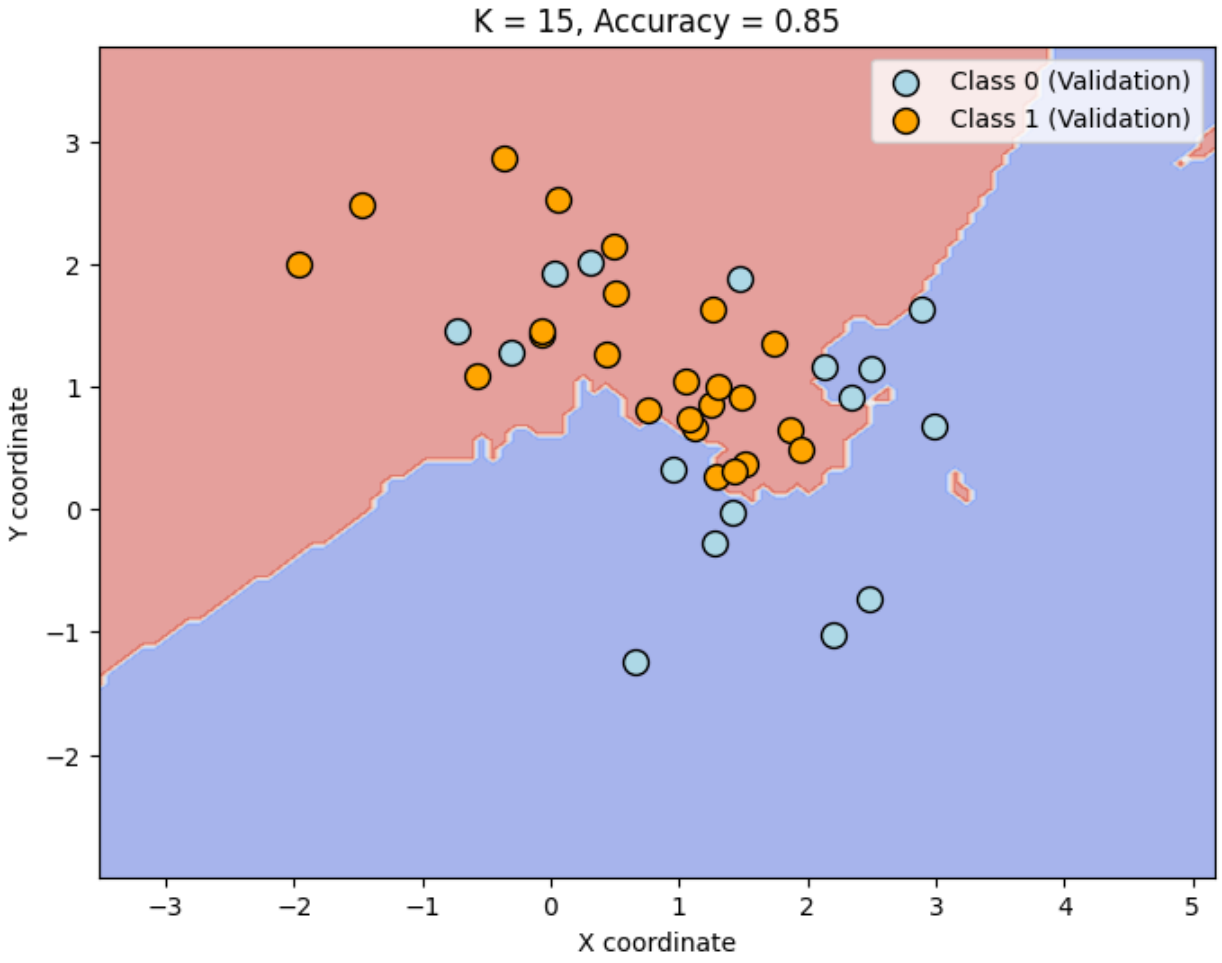


$$\|A-B\|_1 = \left|x_1^{(B)}-x_1^{(A)}\right| + \left|x_2^{(B)}-x_2^{(A)}\right|$$

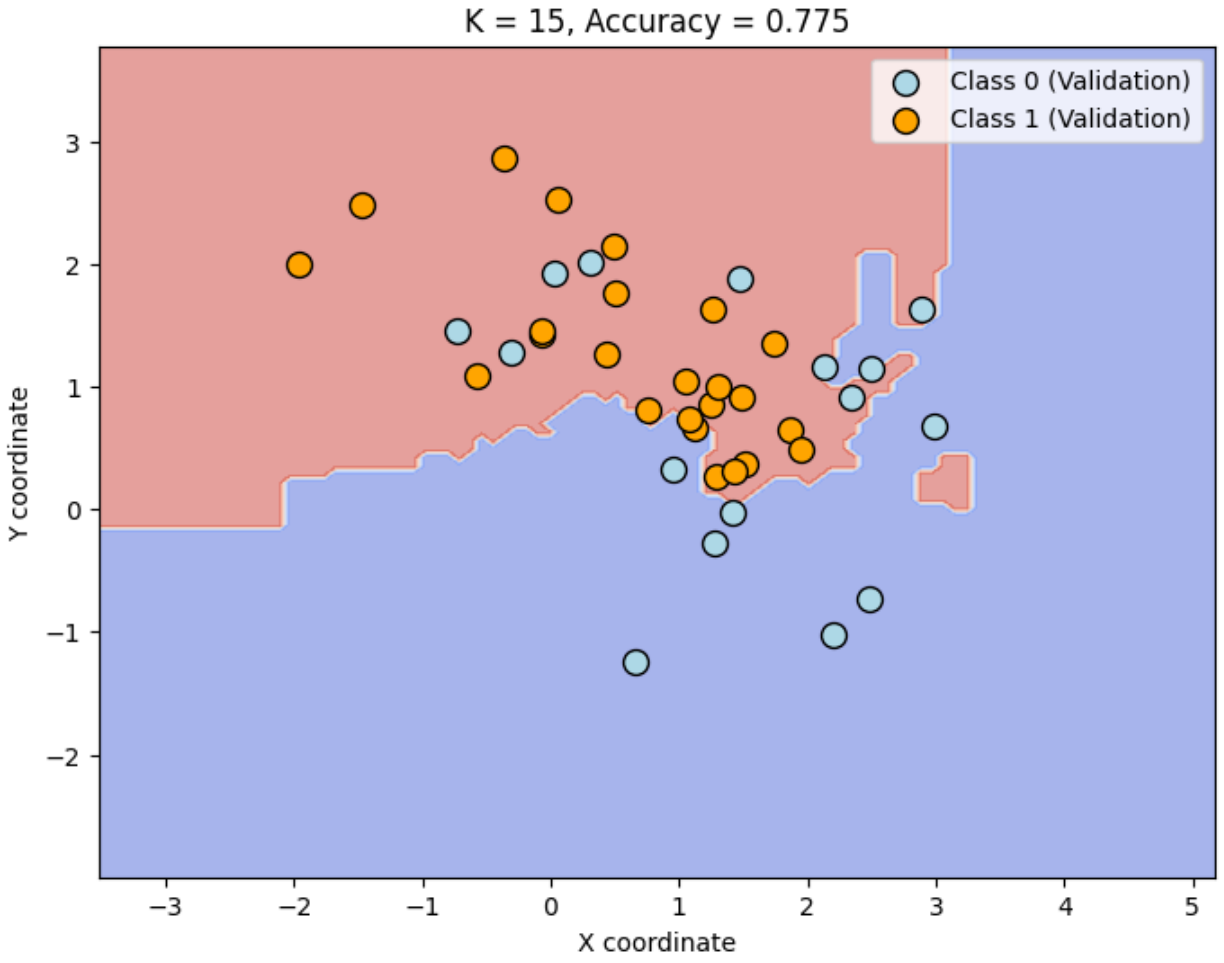
$$= |3-1| + |2-1|$$

$$= 2+1 = 3$$

Decision Boundary



Eucliean Distance



Manhattan Distance

Optimal Model (Revised)

- An optimal k-NN model is one where the choice of **k** and **distance metric** balances the complexity of decision boundaries to achieve high validation accuracy while maintaining good training accuracy, indicating effective generalisation without overfitting or underfitting.

K	Distance	Train Accuracy	Validation Accuracy
1	Manhattan	1	0.775
1	Euclidean	1	0.775
2	Manhattan	1	0.775
2	Euclidean	1	0.775
3	Manhattan	0.892	0.85
3	Euclidean	0.892	0.875
4	Manhattan	0.95	0.85
4	Euclidean	0.958	0.85
5	Manhattan	0.875	0.85
5	Euclidean	0.867	0.85
…	…	…	…

Scaling Effects

A: (170 cm, 60 kg), B: (180 cm, 60 kg)

Distance(A, B) = 10

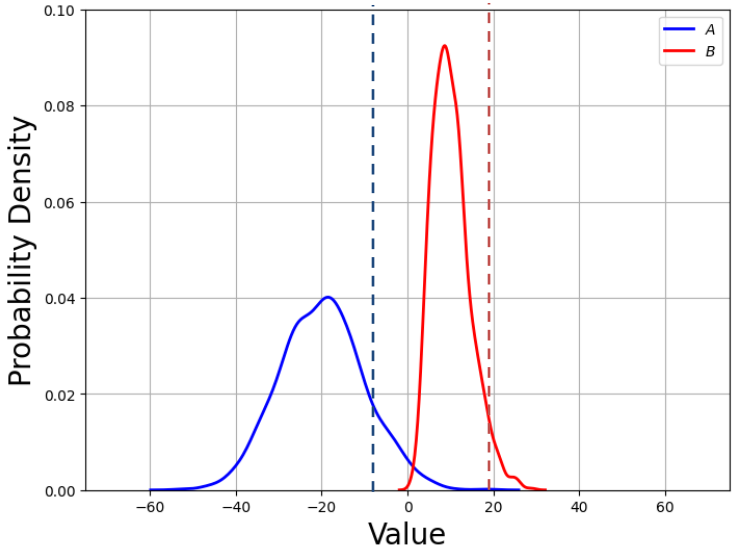
C: (170 cm, 60 kg), D: (170 cm, 70 kg)

Distance(C, D) = 10

- Distance(A, B) = Distance(C, D). Eh?
- In reality, we will able to differentiate A form B quite easilty. This will not be so obvious between C and D.
- Where possible, we must ensure different features are fairly contributing to the machine learning (ML) model.

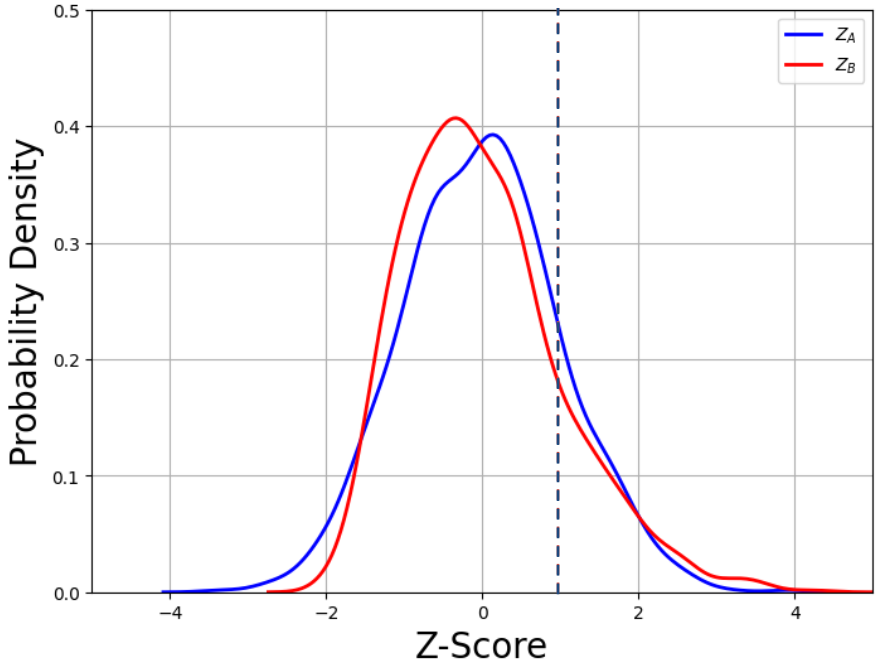
Standardising with Z Scores

$$Z = \frac{x - \mu}{\sigma}$$



$$\mu_A = -19.81$$
$$\sigma_A = 9.88$$

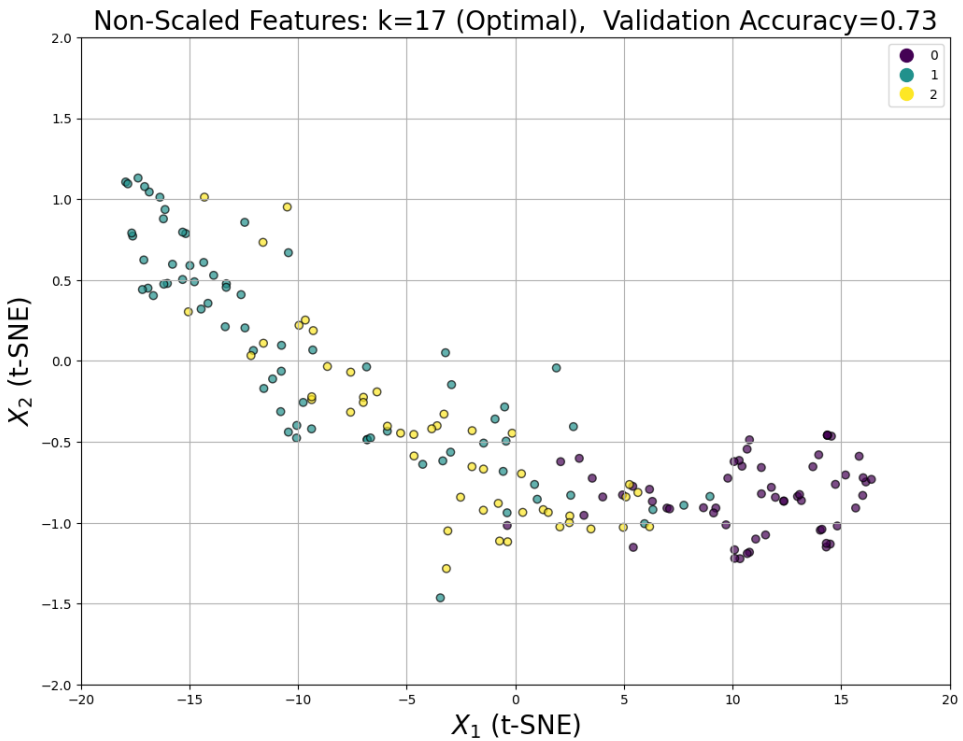
$$\mu_B = 10.16$$
$$\sigma_B = 4.40$$

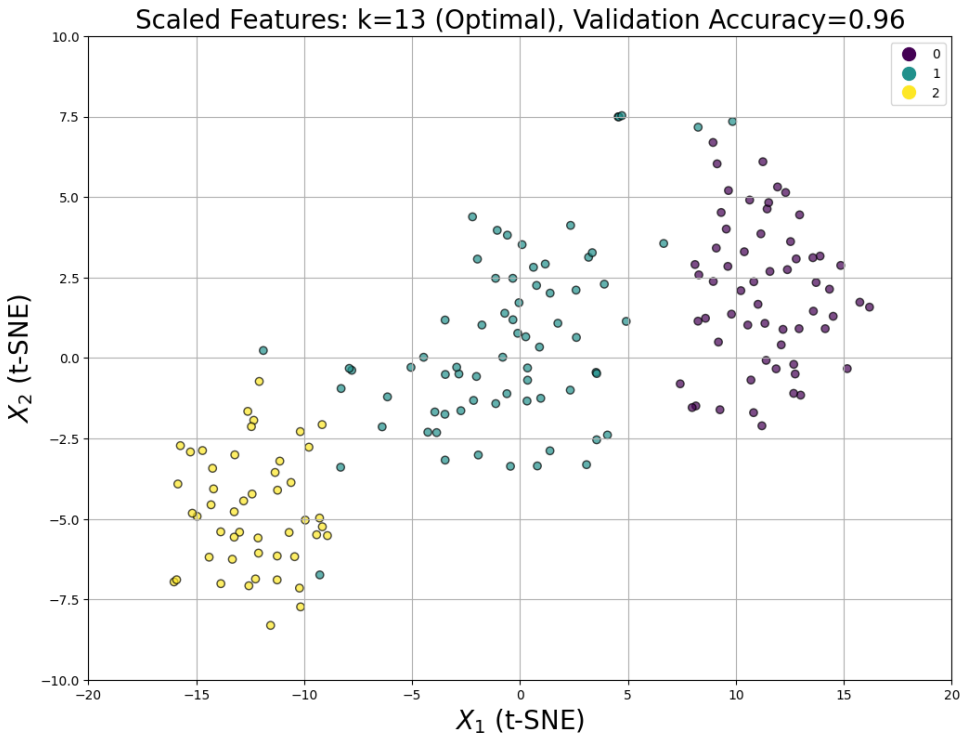


$$Z_A = \frac{-9.93 - (-19.81)}{9.88} = 1$$

$$Z_B = \frac{14.56 - 10.16}{4.40} = 1$$

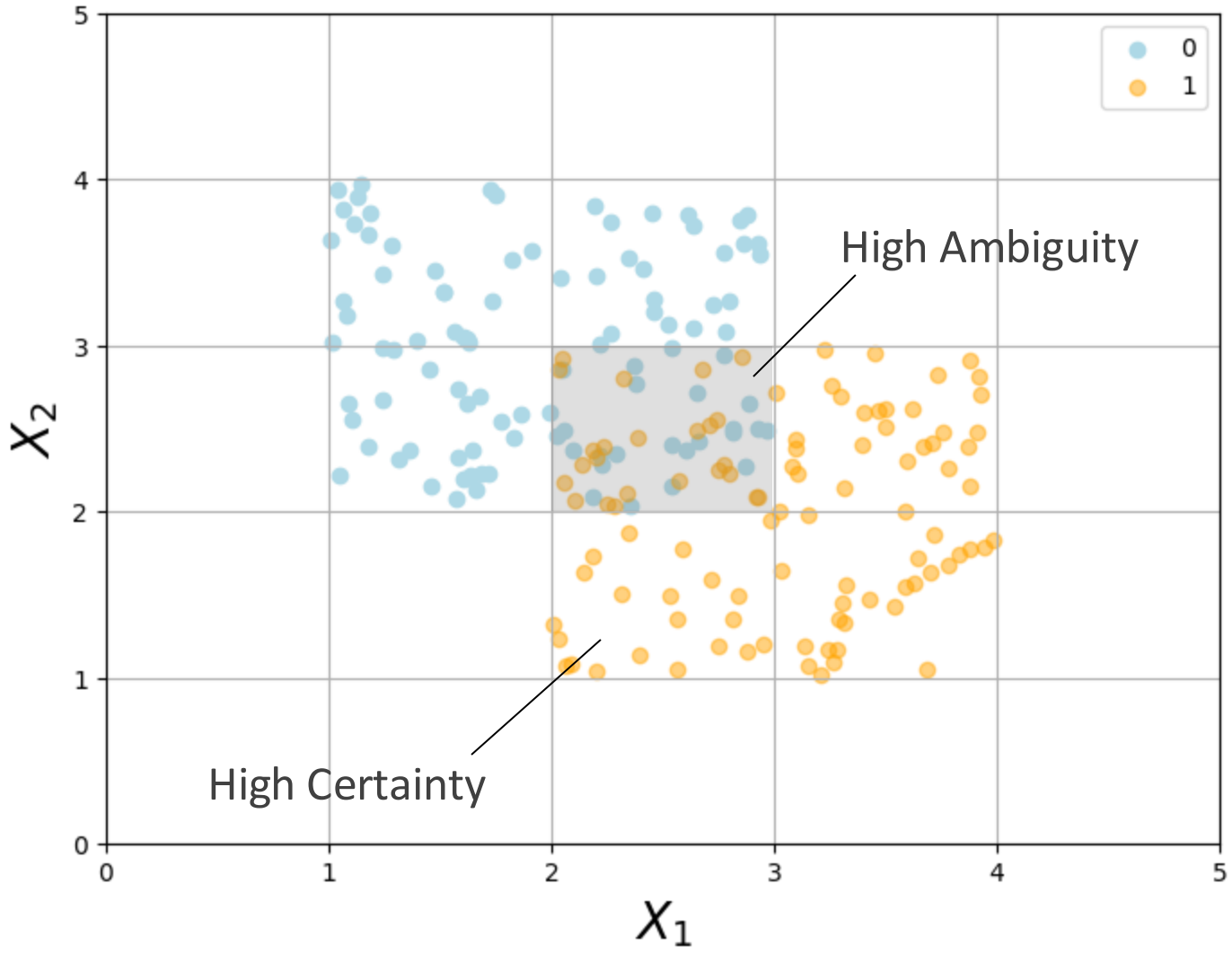
Wine Dataset: Better Prediction Performance with Z-Score Features





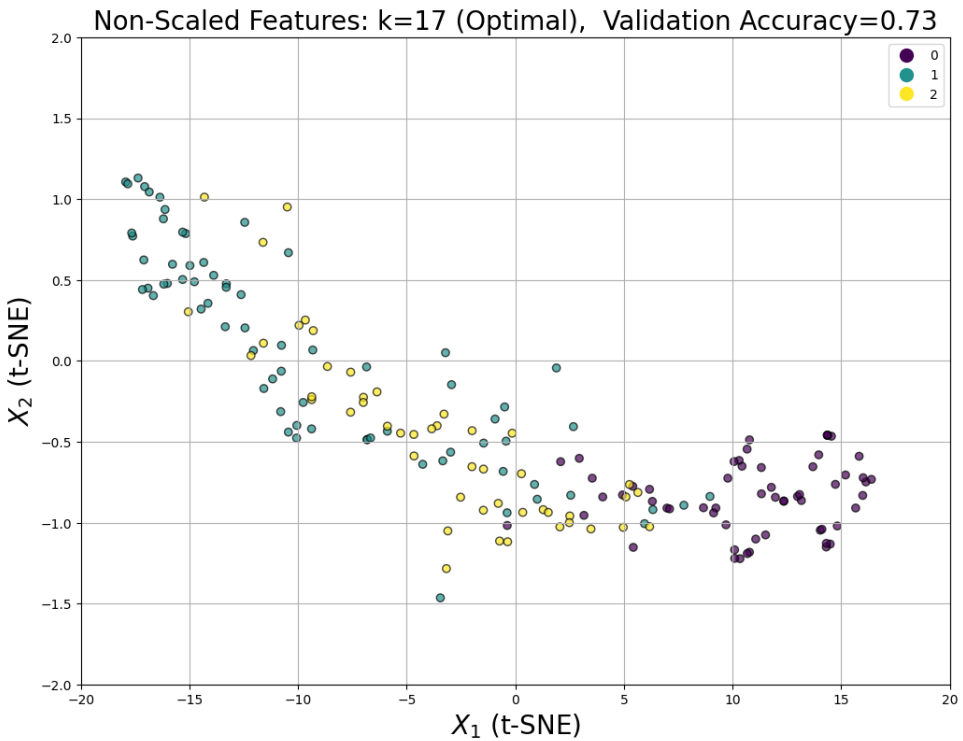
- Wine Dataset: 13 Features. These features are chemical constituents found in wine, specifically from three different cultivars of wine grown in the same region in Italy.
- Standardisation ensures features with different units and scales are contributing fairly into the ML model. This results in better separation between the classes.

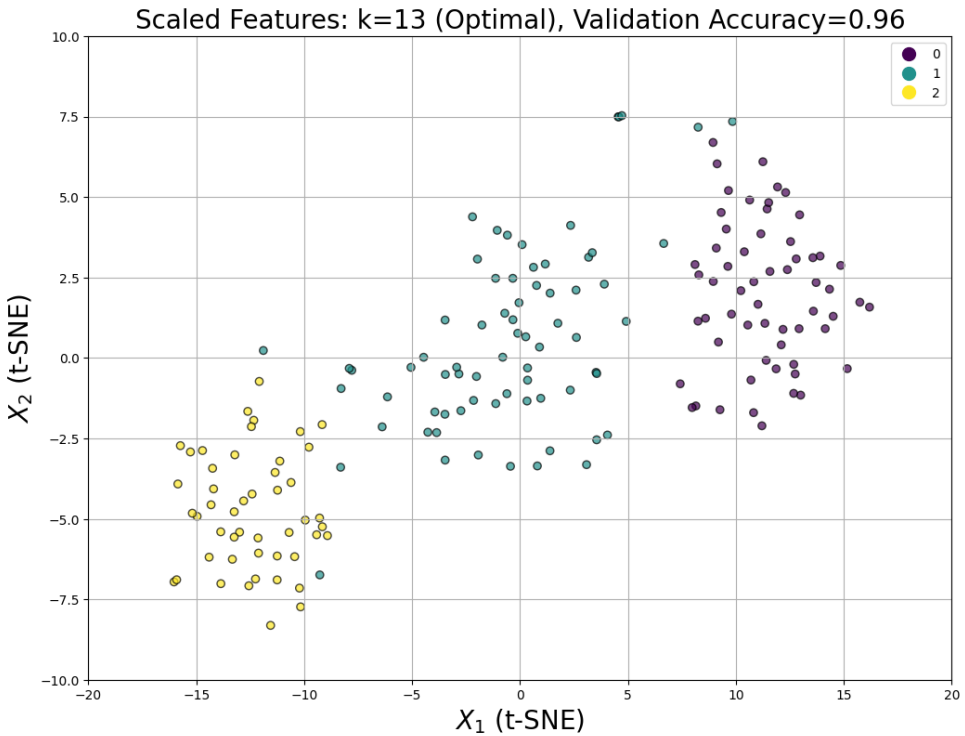
Where Do Errors Come From?



- The area, where the classes are overlapping. is where we are likely to make incorrect predictions. Hence, this is where classification errors are coming from.
- On the contrary, we can predict with high accuracy in the area where this is no (or little) overlap between the classes.

Wine Dataset: Z-Score Features (Revisit)





- When features are not scaled, there are large overlapping areas between the classes. We will make a lot of mistakes, and hence low validation accuracy.
- With standardisation, the classes are well separated. There overlapping areas are small, and hence we can predict with high certainty. This results in a high validation accuracy when we standardise the features.

scikit-learn: StandardScaler

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import numpy as np

# Load the dataset
data = load_wine()
X = data.data
y = data.target

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)

# Create a StandardScaler object
scaler = StandardScaler()

# Fit the scaler to the training data only
scaler.fit(X_train)

# Transform both the training data and testing data
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

- Fit the scaler only on the training data to avoid data leakage, and then use this fitted scaler to transform both the training and test datasets. This ensures that the model is tested on truly unseen data, maintaining the integrity of the test set and enabling an accurate evaluation of model performance.

scikit-learn: t-SNE

```
from sklearn.manifold import TSNE

# t-SNE on the original data
tsne = TSNE(n_components=2, random_state=42)
X_tsne = tsne.fit_transform(X)

# Mapping colors to labels
cmap = plt.cm.viridis
norm = plt.Normalize(y.min(), y.max())

# Figure: t-SNE on Original Data
plt.figure(figsize=(12, 9))
scatter = plt.scatter(X_tsne[:, 0], X_tsne[:, 1], c=y, cmap=cmap, edgecolor='k', alpha=0.7, norm=norm)
plt.xlabel('$X_1$ (t-SNE)', fontsize=20)
plt.ylabel('$X_2$ (t-SNE)', fontsize=20)
plt.axis([-20, 20, -2, 2])
plt.grid()

# Create a legend:
handles = [plt.Line2D([], [], marker='o', color=cmap(norm(i)), linestyle='', markersize=10, label=name[-1]) for i, name in enumerate(target_names)]
plt.legend(handles=handles)
plt.show()
```

- t-SNE is a statistical method for visualizing high-dimensional data by reducing the number of dimensions to 2 or 3 while trying to preserve the relative distances between points. t-SNE is effective for exploring the structure of data and for identifying clusters or groups within the data.

Summary

- The notion of distance (or similarity) underpins how K-Nearest Neighbours (KNN) works. It defines nearness between the data points, which is then used to make predictions. The distance metric will affect KNN's prediction perfomances. The optimal choice is usecase specific and to be found through grid search and cross-validation.
- KNN models, or ML in general, are considered good if they consistently perform well on the training dataset and also on an unseen data. We need to find a model that is neither overfitting nor underderfitting.
- Low K generally leads to a complex decision boundary, i.e. high variance (or low bias). In reverse, high K will lead to a simple one, i.e. high bias (or low variance). We need to find K which will result in high validation accuracy and still maintaining high train accuracy.
- The test dataset is a proxy of an unseen (or real world) data. We only use it for final evaluation of the trained model, never use it during the training. To deploy the model or not, we often base on whether the validation and test performances are consistent with each other.
- Features must be in the same scale, either through standard scaling or normalisation, in order to ensure they are contributing fairly to the KNN (or ML) model. In practice, we always scale our features before training a model, and often this leads to a better prediction performance.